Instructor Handout 1: Introduction to Operating Systems

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 14, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

1.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 1 "Introduction" (what an OS is, pp. 3–47; dual-mode operation, pp. 20–23; system calls) and Ch. 2 "System Structures" (services, system calls, system programs, structure designs, pp. 49–89). *Page-verified* — index built 2026-10-14. This is your primary anchor.
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 2 §2.2 (evolution: batch, multiprogramming, time-sharing; pp. 55–64), Ch. 3 (§3.3 process description / PCB, pp. 107–152), Ch. 1 (interrupts, I/O techniques). *Page-verified.*
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e** — Ch. 1 (basic OS concepts, files and inodes) and Ch. 10 (system calls: entering/exiting the kernel, parameter passing). Optional depth for the concrete kernel view. *Page-verified.*
- **Plan the running thread before class.** Choose one command you can trace end-to-end without hesitation — `cp notes.txt backup.txt` works well because it is short, familiar, and exercises open/read/write/close. Write the five-step trace on the board from memory first; the numbers students care about (how many mode switches, where the driver enters) must be right the first time.
- **Decide your concrete machine.** Name one real OS students use (Linux, Windows, macOS) and refer to it when discussing structure, so "monolithic vs. modular" is not purely abstract.
- **Prerequisite check.** Students have seen CPU, registers, memory, and I/O in CS401. Spend at most two minutes recalling the memory hierarchy and interrupts; do not re-teach them here.

1.2 Opening: The Question This Course Answers

Open by typing (or saying) the command `cp notes.txt backup.txt` and asking the class what happens between pressing Enter and the file appearing. Accept a few answers, then point out that everything they named is one layer of a stack: shell, kernel, system-call boundary, device driver, controller. Only some of those are "the operating system." This gap — between the command and the machine — is the subject of the whole course.

Teaching note: Resist defining the OS immediately. Let the class generate a messy list of guesses first; the definition lands better when it resolves a confusion they just voiced. Two minutes of guessing is worth ten of telling.

1.3 What Is an Operating System?

Give the three definitions together, not one at a time, and make students say which view answers which question.

Definition (The OS as resource manager). *Manages CPU, memory, storage, and I/O — deciding who gets what, and when.*

Definition (The OS as control program). *Controls the execution of user programs to prevent errors and improper use — the referee.*

Definition (The OS as extended machine). *Hides messy hardware behind clean abstractions (files, processes, sockets).*

Walk the `cp` thread through the resource-manager view: the CPU runs `cp`, memory holds its buffer, the disk stores the file, the I/O path moves the bytes. The OS multiplexes all four across competing processes. Then switch to the extended-machine view: the disk is really (cylinder, head, sector) blocks, but nobody writes code against that; they write against a **file**.

Teaching note: The single most useful sentence in this section: "abstraction is what makes portability possible." A program that knew disk geometry would break on the next disk. Say it explicitly; students tend to treat abstraction as tidiness rather than as the enabling condition for software that survives hardware change.

1.4 The User/Kernel Boundary

Draw the four layers on the board (Figure 1.1): user applications, the system-call interface, the kernel, the hardware. Then introduce the **mode bit**: user mode (1) vs. kernel mode (0). Privileged instructions are legal only in kernel mode.

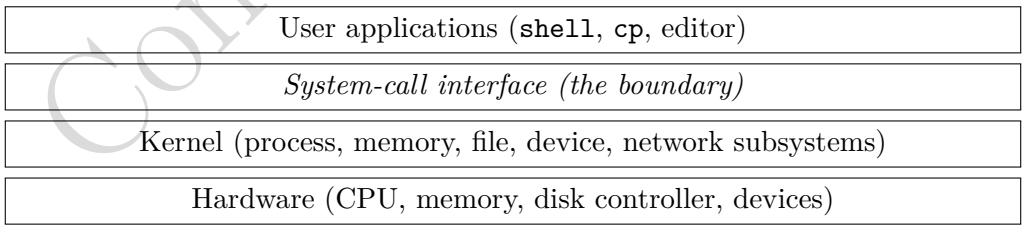


Figure 1.1: The user/kernel boundary. User programs reach the kernel only through the system-call interface.

Teaching note: A crisp way to frame it: the **system call** is the only legal door between the two worlds. Everything a program wants from hardware — every byte, every CPU cycle it does not already own — must pass through that door, and the kernel gets to inspect the request. This is why "just access the hardware" is not an option for user code, and why that restriction is a feature, not a limitation.

Concepts That Click

1.5 Evolution: Why Each Feature Exists

Teach the evolution as a chain of failures, not a timeline to memorize. Each step exists because the previous design failed a real workload.

- **Serial processing** (1940s–50s): one job at a time; a human is the scheduler. Failure: the CPU idles while the operator mounts tapes — terrible utilization.
- **Batch OS**: a **resident monitor** runs jobs from a batch automatically, driven by a **job-control language**. Failure fixed: less operator overhead.
- **Multiprogramming**: keep several jobs in memory; switch on I/O waits. Objective: **CPU utilization**. **Spooling** buffers I/O on disk so slow devices overlap the CPU.
- **Time-sharing**: switch fast enough that every user has an interactive session. Objective: **response time**. This is the birth of the shell and the prompt (CTSS, Multics, UNIX).
- **Real-time**: correctness depends on *when*. **Hard** real-time — a missed deadline is a failure; **soft** real-time — a missed deadline degrades quality.

> **Teaching note:** The exam-worthy contrast: multiprogramming maximizes *utilization*; time-sharing minimizes *response time*. These can conflict. Ask the class which one a batch payroll run wants (utilization) and which a login prompt wants (response). Getting them to say the two objectives out loud is the whole point.

1.6 System Protection

Present the four mechanisms as a single coordinated contract, not four separate facts:

- **Dual-mode operation** — the mode bit restricts *what* may be executed.
- **System call** — the controlled entry point into the kernel.
- **Memory protection** — base/limit registers or page tables restrict *what memory* may be touched.
- **Timer interrupt** — guarantees the kernel regains control.

> **Teaching note:** The timer is the one students forget, and it is the one that makes preemption possible. Ask: "Suppose we have dual-mode operation but no timer. A user process loops forever in user mode. What happens?" The answer — nothing; the kernel never regains the CPU — shows why the timer is non-negotiable. Then recall the Socratic Check on the slide: the timer is the exact mechanism.

1.7 Operating-System Structure

Move through the five structures as a spectrum from least to most structured, each with a named cost.

Concepts That Click

- **Monolithic / simple**: all services in one address space; direct calls are fast but there is no internal protection, so a subsystem bug can take down the kernel. MS-DOS had no separation at all.
- **Layered**: level N uses only level $N-1$; easier to design and debug, but layers are hard to define and add call overhead (THE OS, six layers).
- **Microkernel**: only IPC, basic scheduling, and low-level memory in the kernel; the rest are user-space servers (Mach, QNX, L4). Reliable, secure, portable; message-passing overhead.
- **Modular**: core kernel plus loadable modules with defined interfaces (Linux, Solaris, macOS). Near-monolithic speed with microkernel-like flexibility.
- **Virtual machine**: a hypervisor presents virtual hardware; each guest runs its own OS (VMware, Xen, KVM). Stereo isolation; complex, expensive I/O virtualization.

> **Teaching note:** If pressed for time, prioritize microkernel vs. monolithic vs. modular — those three are the standard exam contrasts. The virtual-machine row is worth one sentence and can be deferred to a later lecture on virtualization, which this syllabus does not formally include but a keen class will ask about.

1.8 OS Services, System Calls, and System Programs

Split services by audience (user-facing vs. system-efficiency), then spend the bulk of the remaining time on system calls, because that is where the assessment lives.

System-call families: process control (`fork`, `exec`, `wait`, `exit`); file management (`open`, `read`, `write`, `close`, `unlink`); device management; information maintenance; communication; protection. Stress that POSIX/Win32 APIs *wrap* system calls — one library call may issue several.

The running thread, front to back:

1. `open("notes.txt", O_RDONLY)` — system call; mode switch; kernel resolves the path and checks permission.
2. A **file descriptor** returns; the kernel has created an **FCB** entry.
3. `read`/`write` move bytes through a user buffer; each call is a separate trap.
4. The kernel hands block requests to a **device driver** → disk **controller**.
5. `close` releases the descriptor; success is silent.

Parameter passing: registers (simple, limited), a parameter block whose address is in a register (Linux/Solaris), or the stack.

System programs: shells, compilers, file managers, editors, `/bin` utilities — user-level tools built on system calls; the visible face of the OS, but not the kernel.

> **Teaching note:** Do the trace live, counting mode switches on the board (five for this command). Then ask: "The programmer wrote five calls. How much OS code ran behind them?" The answer — path resolution, permissions, caching, scheduling, drivers — is the payoff of the thread: it makes the "extended machine" concrete and shows why the abstraction earns its overhead.

Concepts That Click

1.9 Where Students Get Stuck

- **"The OS is the kernel."** Correct that with the three-views framing + the kernel-vs-shell distinction in the Socratic Check.
- **Forgetting the timer.** They will say dual-mode is enough. Force the infinite-loop scenario.
- **Confusing multiprogramming and time-sharing.** Anchor each to its objective (utilization vs. response).
- **Microkernel as "just a smaller kernel."** The key is *isolation*: crash containment and the IPC cost that buys it.
- **System call vs. API.** The API is the programmer's contract; the system call is what the kernel runs. A single API call may trigger several system calls.

1.10 Common Misconceptions to Preempt

- "Protection is a software policy." No — it is a hardware/OS contract (mode bit, base/limit, timer). Without hardware, no software can enforce it.
- "A faster quantum is always better for responsiveness." Diminishing returns; overhead grows and throughput collapses (see Assignment Q4).
- "The shell is part of the kernel because it feels like the OS." It is a system program in user mode.

1.11 Exit Ticket / Comprehension Checks

Use the three Socratic Checks from the deck, in order: (1) is a browser/driver/firmware an OS; (2) the infinite-loop timer question; (3) is the shell part of the OS. The third doubles as the bridge to the next lecture's idea that the OS manages processes, not commands.

1.12 Readings to Prepare Next Time

- **Silberschatz et al., *Operating System Concepts*, 8e, Ch. 3 §3.1–3.3 (pp. 101–116)** — process concept, process state diagram, PCB, schedulers, context switch. *(Page-verified.)*
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e, Ch. 3 (pp. 107–152)** — the process model as the concrete counterpart. *(Page-verified.)*
- Skim **Bovet & Cesati, *Understanding the Linux Kernel*, 3e, Ch. 3 (pp. 79–130)** for the Linux 'task_struct' and process switch, which reappears throughout the course. *(Page-verified.)*

Concepts That Click